

ReGo - Quick Start Guide for Developers

Get up and running in 30 minutes

Day 1 Setup Checklist

Prerequisites (Install These First)

- ☐ **Node.js 20.x LTS** - <https://nodejs.org>
- ☐ **PostgreSQL 15.x** - <https://www.postgresql.org/download/>
- ☐ **Git** - <https://git-scm.com>
- ☐ **VS Code** - <https://code.visualstudio.com>
- ☐ **Postman** - <https://www.postman.com/downloads/>

VS Code Extensions (Recommended)

- ESLint
- Prettier
- GitLens
- PostgreSQL (by Chris Kolkman)
- Thunder Client (alternative to Postman)
- ES7+ React/Redux snippets

Project Setup

1. Create Project Structure

```
bash

# Create main project folder
mkdir rego-travel-system
cd rego-travel-system

# Create backend and frontend folders
mkdir backend frontend
```

2. Backend Setup (Node.js + Express)

```
bash
```

```
cd backend
```

```
# Initialize npm
```

```
npm init -y
```

```
# Install dependencies
```

```
npm install express cors dotenv bcrypt jsonwebtoken
```

```
npm install pg prisma @prisma/client
```

```
npm install multer @azure/storage-blob @azure/ai-form-recognizer
```

```
npm install winston joi express-validator
```

```
npm install axios nodemailer
```

```
# Install dev dependencies
```

```
npm install --save-dev nodemon jest supertest eslint prettier
```

```
# Create folder structure
```

```
mkdir src
```

```
cd src
```

```
mkdir controllers routes services middleware models config utils
```

```
cd ..
```

```
# Create entry point
```

```
touch server.js
```

```
# Create environment file
```

```
touch .env
```

Package.json scripts:

```
json
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "jest --watchAll"
  }
}
```

Basic server.js:

```
javascript
```

```

const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Routes
app.get('/health', (req, res) => {
  res.json({ status: 'OK', message: 'ReGo API is running' });
});

// Start server
const PORT = process.env.PORT || 8080;
app.listen(PORT, () => {
  console.log(`🚀 Server running on port ${PORT}`);
});

```

.env file:

```

bash

NODE_ENV=development
PORT=8080

# Database
DATABASE_URL=postgresql://postgres:password@localhost:5432/rego_db

# JWT
JWT_SECRET=your-secret-key-change-in-production
JWT_EXPIRE=15m
REFRESH_TOKEN_SECRET=your-refresh-secret
REFRESH_TOKEN_EXPIRE=7d

# Azure (Add later when you get credentials)
AZURE_STORAGE_ACCOUNT_NAME=
AZURE_STORAGE_ACCOUNT_KEY=
AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT=
AZURE_DOCUMENT_INTELLIGENCE_KEY=

```

3. Frontend Setup (React)

```

bash

```

```
cd ../frontend
```

```
# Create React app
```

```
npx create-react-app .
```

```
# Install dependencies
```

```
npm install @mui/material @mui/icons-material @emotion/react @emotion/styled
```

```
npm install react-router-dom redux @reduxjs/toolkit react-redux
```

```
npm install axios formik yup
```

```
npm install react-query date-fns chart.js react-chartjs-2
```

```
# Install dev dependencies
```

```
npm install --save-dev @testing-library/react @testing-library/jest-dom
```

```
# Create folder structure
```

```
cd src
```

```
mkdir components pages features hooks services routes utils styles store
```

```
cd components
```

```
mkdir common layout features
```

```
cd ../../
```

Update src/index.js:

```
javascript
```

```
import React from 'react';
```

```
import ReactDOM from 'react-dom/client';
```

```
import { BrowserRouter } from 'react-router-dom';
```

```
import { Provider } from 'react-redux';
```

```
import { store } from './store/store';
```

```
import App from './App';
```

```
import './styles/globalStyles.css';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(  
  <React.StrictMode>  
    <Provider store={store}>  
      <BrowserRouter>  
        <App />  
      </BrowserRouter>  
    </Provider>  
  </React.StrictMode>  
);
```

Create .env:

```
bash
```

```
REACT_APP_API_URL=http://localhost:8080/api/v1
```

```
REACT_APP_ENV=development
```

4. Database Setup

```
bash
```

```
# Open PostgreSQL terminal
```

```
psql -U postgres
```

```
# Create database
```

```
CREATE DATABASE rego_db;
```

```
# Connect to database
```

```
\c rego_db
```

```
# Run the schema creation SQL
```

```
# (Copy from the technical documentation)
```

Or use Prisma:

```
bash
```

```
cd backend
```

```
# Initialize Prisma
```

```
npx prisma init
```

```
# Edit prisma/schema.prisma with your models
```

```
# Then run:
```

```
npx prisma generate
```

```
npx prisma migrate dev --name init
```

Testing Your Setup

Backend Test

```
bash
```

```
cd backend  
npm run dev
```

In another terminal:

```
curl http://localhost:8080/health
```

Expected response:

```
# {"status": "OK", "message": "ReGo API is running"}
```

Frontend Test

```
bash
```

```
cd frontend
```

```
npm start
```

Browser should open to http://localhost:3000

Database Test

```
bash
```

In PostgreSQL terminal:

```
\c rego_db
```

```
\dt
```

Should show all your tables

Common Commands Reference

Git Commands

```
bash
```

Clone repository

`git clone <repo-url>`

Create new branch

`git checkout -b feature/your-feature-name`

Check status

`git status`

Add files

`git add .`

Commit changes

`git commit -m "Your commit message"`

Push to remote

`git push origin feature/your-feature-name`

Pull latest changes

`git pull origin main`

Merge main into your branch

`git checkout your-branch`

`git merge main`

npm Commands

`bash`

Install all dependencies

npm install

Install specific package

npm install package-name

Install dev dependency

npm install --save-dev package-name

Uninstall package

npm uninstall package-name

Run script

npm run script-name

Check for outdated packages

npm outdated

Update packages

npm update

Database Commands

bash

```
# PostgreSQL
```

```
psql -U postgres -d rego_db
```

```
# List databases
```

```
\l
```

```
# List tables
```

```
\dt
```

```
# Describe table
```

```
\d table_name
```

```
# Run SQL file
```

```
\i /path/to/file.sql
```

```
# Exit
```

```
\q
```

```
# Backup database
```

```
pg_dump -U postgres rego_db > backup.sql
```

```
# Restore database
```

```
psql -U postgres rego_db < backup.sql
```

Troubleshooting

Port Already in Use

```
bash
```

```
# Find process using port 8080
```

```
lsof -i :8080
```

```
# Kill process
```

```
kill -9 <PID>
```

```
# Or change port in .env file
```

Database Connection Error

```
bash
```

Check if PostgreSQL is running

```
sudo service postgresql status
```

Start PostgreSQL

```
sudo service postgresql start
```

Check connection string in .env

```
DATABASE_URL=postgresql://username:password@localhost:5432/database_name
```

React App Not Starting

```
bash
```

Clear cache

```
npm cache clean --force
```

Delete node_modules and reinstall

```
rm -rf node_modules package-lock.json
```

```
npm install
```

Check for port conflicts

Default React port is 3000

CORS Issues

Backend - Add CORS middleware:

```
javascript
```

```
const cors = require('cors');
```

```
app.use(cors({  
  origin: 'http://localhost:3000',  
  credentials: true  
}));
```



Code Templates

API Route Template

```
javascript
```

```
// routes/travelRequestRoutes.js
```

```
const express = require('express');
```

```
const router = express.Router();
```

```
const authMiddleware = require('../middleware/auth');
```

```
const { createTravelRequest, getTravelRequests } = require('../controllers/travelRequestController');
```

```
router.post('/', authMiddleware, createTravelRequest);
```

```
router.get('/', authMiddleware, getTravelRequests);
```

```
module.exports = router;
```

Controller Template

```
javascript
```

```
// controllers/travelRequestController.js
```

```
const { PrismaClient } = require('@prisma/client');
```

```
const prisma = new PrismaClient();
```

```
exports.createTravelRequest = async (req, res) => {
```

```
  try {
```

```
    const { travelType, purpose, destinationCity, departureDate, returnDate } = req.body;
```

```
    const request = await prisma.travelRequest.create({
```

```
      data: {
```

```
        userId: req.user.userId,
```

```
        travelType,
```

```
        purpose,
```

```
        destinationCity,
```

```
        departureDate: new Date(departureDate),
```

```
        returnDate: new Date(returnDate),
```

```
        status: 'DRAFT'
```

```
      }
```

```
    });
```

```
    res.status(201).json({
```

```
      success: true,
```

```
      data: request,
```

```
      message: 'Travel request created successfully'
```

```
    });
```

```
  } catch (error) {
```

```
    console.error('Error creating travel request:', error);
```

```
    res.status(500).json({
```

```
      success: false,
```

```
      error: { message: 'Internal server error' }
```

```
    });
```

```
  }
```

```
};
```

```
exports.getTravelRequests = async (req, res) => {
```

```
  try {
```

```
    const requests = await prisma.travelRequest.findMany({
```

```
      where: { userId: req.user.userId },
```

```
      orderBy: { createdAt: 'desc' }
```

```
    });
```

```
    res.json({
```

```
      success: true,
```

```
      data: { requests }
```

```
    });
```

```
  } catch (error) {
```

```
console.error('Error fetching requests:', error);
res.status(500).json({
  success: false,
  error: { message: 'Internal server error' }
});
}
};
```

React Component Template

jsx

```
// pages/Dashboard/Dashboard.jsx
import React, { useEffect } from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { Box, Typography, Grid } from '@mui/material';

const Dashboard = () => {
  const dispatch = useDispatch();
  const { user } = useSelector(state => state.auth);
  const { loading, error } = useSelector(state => state.dashboard);

  useEffect(() => {
    // Fetch dashboard data
    dispatch(fetchDashboardData());
  }, [dispatch]);

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4" gutterBottom>
        Welcome, {user.firstName}!
      </Typography>

      <Grid container spacing={3}>
        { /* Add your dashboard content here */ }
      </Grid>
    </Box>
  );
};

export default Dashboard;
```

Redux Slice Template

javascript

```
// features/auth/authSlice.js
```

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
```

```
import authService from '../services/authService';
```

```
export const login = createAsyncThunk(
  'auth/login',
  async ({ email, password }, { rejectWithValue }) => {
    try {
      const response = await authService.login(email, password);
      return response.data;
    } catch (error) {
      return rejectWithValue(error.response.data);
    }
  }
);
```

```
const authSlice = createSlice({
  name: 'auth',
  initialState: {
    user: null,
    token: null,
    loading: false,
    error: null
  },
  reducers: {
    logout: (state) => {
      state.user = null;
      state.token = null;
    }
  },
  extraReducers: (builder) => {
    builder
      .addCase(login.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(login.fulfilled, (state, action) => {
        state.loading = false;
        state.user = action.payload.user;
        state.token = action.payload.token;
      })
      .addCase(login.rejected, (state, action) => {
        state.loading = false;
        state.error = action.payload.error.message;
      });
  }
});
```

```
});
```

```
export const { logout } = authSlice.actions;
```

```
export default authSlice.reducer;
```

First Week Goals

Day 1-2: Setup

- ☐ Install all prerequisites
- ☐ Create project structure
- ☐ Set up Git repository
- ☐ Initialize npm projects
- ☐ Create database

Day 3-4: Authentication

- ☐ Create User model
- ☐ Implement registration endpoint
- ☐ Implement login endpoint
- ☐ Add JWT middleware
- ☐ Create login page (frontend)

Day 5-6: Basic UI

- ☐ Create layout components
- ☐ Set up routing
- ☐ Create dashboard page
- ☐ Add navigation

Day 7: Testing & Demo Prep

- ☐ Test all features
- ☐ Fix bugs
- ☐ Prepare demo
- ☐ Document what's done

Pro Tips

1. **Commit Often:** Make small, frequent commits with clear messages
2. **Branch Strategy:** Use feature branches, don't commit directly to main
3. **Test Locally:** Always test before pushing
4. **Use Console.log:** Debug with console.log, remove before committing
5. **Read Errors:** Error messages usually tell you exactly what's wrong

6. **Google It:** If stuck, Google the error message
 7. **Ask Team:** Don't struggle alone, ask for help
 8. **Take Breaks:** Step away when frustrated, fresh eyes help
 9. **Comment Code:** Add comments for complex logic
 10. **Follow Conventions:** Use consistent naming and formatting
-

Getting Help

When Stuck:

1. **Read the error message** carefully
2. **Google the error** with "Node.js" or "React"
3. **Check documentation** for the library you're using
4. **Ask ChatGPT** or Claude for help
5. **Ask your team** in Slack/Discord
6. **Check Stack Overflow**

Useful Resources:

- **Node.js Docs:** <https://nodejs.org/docs>
 - **React Docs:** <https://react.dev>
 - **Material-UI:** <https://mui.com>
 - **Prisma Docs:** <https://www.prisma.io/docs>
 - **Express Docs:** <https://expressjs.com>
-

Team Contacts

- **Backend Lead (Dev 1):** [Contact info]
 - **Frontend Lead (Dev 2):** [Contact info]
 - **Travel Module (Dev 3):** [Contact info]
 - **AI Module (Dev 4):** [Contact info]
 - **QA (Dev 5):** [Contact info]
-

Daily Standup Template

What I did yesterday:

What I'll do today:

Blockers:

Good luck! Remember: **Everyone starts somewhere. Don't be afraid to ask questions!** 